Edge Cases: Clustered Riders, Empty Zones and Other Surprises

Oct 23, 2026

Many scooters in one place

Clustering is a coverage problem: ten scooters at one corner cover the same gates ten times and leave the next street uncovered, so each staging node needs a cap and the extras must be pushed outward.

Why it happens

- Riders return to the same hub to restock, and idle scooters stay where their last delivery ended.
- One corner is the best-known spot, so riders drift to it.
- Demand moves, but scooters follow it a few minutes late and bunch up.

What it costs

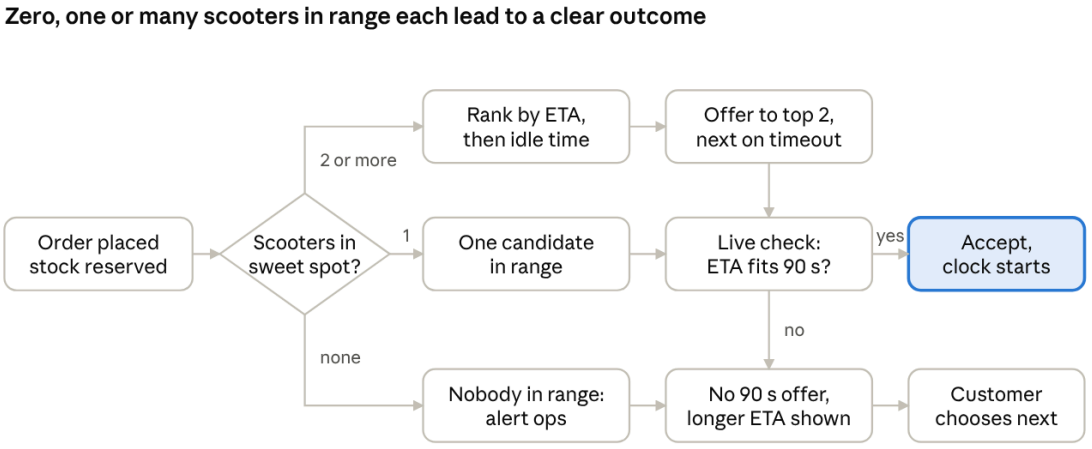
- Redundant coverage in one pocket and empty pockets next to it.
- Idle riders you pay for, plus uneven earnings between riders.
- A longer wait for the customers in the uncovered pockets.

What the system does

1. **Node capacity.** Each staging node has a maximum scooter count from the placement optimizer. Extra scooters are assigned to the next-best node.
2. **Marginal coverage score.** A scooter's value at a node is the demand-weighted number of gates it would cover that no other idle scooter already covers within the 55 s budget. Each idle scooter goes to the node with the highest marginal value (greedy set cover).
3. **Offer to few, not all.** For a new order, rank the cluster by p90 ETA, then by longest idle time for fairness, and offer to the top one or two. Notifying everyone makes riders race and wastes their time.
4. **Rebalance after every pickup.** When a cluster is above its cap, one scooter gets a relocation request to the highest-value free node.
5. **Cooldown.** A scooter is not moved again for a set period, so the fleet does not thrash between nodes.

Repositioning is paid time. A rider is never penalised for moving, or for refusing a move that needs an unsafe route.

No scooter nearby



dispatcher decision flow · 3 branches

With no scooter in range, the dispatcher withdraws the 90-second promise, shows the real ETA, lets the customer choose, and alerts ops about the dead zone. A failed live check ends in the same place, so an empty zone and a bad route are handled the same way.

Rebalancing idle scooters

Rebalancing is an assignment problem solved every few minutes: choose how many scooters each staging node should hold, then move scooters there with the least travel.

```
\max \sum_{g} d_g \, c_g \quad \text{subject to} \quad c_g \le \sum_{n \in N(g)} x_n, \;\; \sum_n x_n = S, \;\; x_n \le \text{cap}_n
```

Here d_g is forecast demand at gate g, c_g is 1 if the gate is covered, x_n is the scooter count at node n, N(g) is the set of nodes that reach gate g within the 55 s budget, and S is the number of idle scooters.

1. **Forecast.** Demand per gate and hour from past orders (rolling average by hour and weekday).
2. **Target counts.** Solve the objective above with greedy set cover, or an ILP in OR-Tools for the exact answer.
3. **Match scooters to targets.** Hungarian algorithm, minimising relocation time.
4. **Stock-aware.** A scooter with low stock goes to a hub first. A full scooter sitting in a dead zone swaps places with an empty one.
5. **Only move for a real gain.** Run on a timer and on events (order taken, scooter offline), and relocate only if coverage improves by more than a set threshold, so scooters do not shuffle for nothing.
6. **Dead-zone alert.** A gate with no coverage for a set time raises an alert for ops, who can add a node or a scooter.

Scenario catalog

Every row below has the same rule: if the checks fail, the system withdraws the 90-second promise instead of stretching it.

Scenario	What the system does	Fallback	Watch
Ten scooters at one node	Node cap, marginal-coverage move, offer to top 1-2 only	Relocate extras	Cluster index per node
No scooter in any sweet spot	No 90 s offer; show a longer ETA	Nearest scooter with customer consent, or wait with a timeout	Empty-zone minutes
One scooter in range, two orders at once	First order wins on ETA; second is re-checked after the first leaves	Longer ETA or decline	Orders lost to no coverage
Two customers want the last unit	Reservation with TTL; loser sees alternatives	Offer another scooter	Stock-race rate
Burst of orders to one gate	Cap concurrent 90 s orders per gate	Queue with honest ETA	Orders per gate per minute
Scooter in range but low on stock	Removed from candidates	Send to hub	Stock-out rate
Rider ignores an offer	Short offer timeout, then re-offer before any acceptance	Next candidate	Ignored-offer rate
Rider goes offline or battery dies before acceptance	Release reservation and rematch	Longer ETA	Pre-accept drop-outs
Breakdown or accident after acceptance	Safety first; tell the customer; send nearest scooter	Refund or credit; log the miss	Missed-promise rate
GPS jump or spoofed location	Map matching, speed plausibility, two pings to trigger	Drop the ping, flag the device	GPS anomaly count
Rain, traffic jam or road closure	Raise edge weights, shrink sweet spots	Suspend 90 s in the zone	Zones suspended
Gate closed or entry refused	Use the pre-agreed gate pin, call the customer	Hand over at nearest open point	Gate failure rate
Customer unreachable at gate	Wait window, call, then end per policy	Waste and refund policy	No-show rate
Wrong gate pin or customer moves	Confirm the pin before payment; allow changes only before acceptance	Re-quote if the gate changes	Pin changes
Temperature breach in a scooter	Quarantine the lot, pull the scooter from search	Refund or replace	Failed temperature checks

Scenario	What the system does	Fallback	Watch
Payment fails after reserve	Release stock, no acceptance	Retry payment	Payment failures
App crash or no network at gate	Server timer keeps running; rider events queue offline	Reconcile later with signed events	Offline events
Hub congestion at restock	Staggered restock slots	Second hub or a longer restock window	Restock wait
Demand spike (rain, event)	Narrow the menu, raise the buffer	Pause 90 s for the zone	Spike-hour miss rate
Rider refuses an unsafe route	No penalty; rematch	Longer ETA	Refusals
Duplicate or stale commit	Idempotency key and fencing token	Reject the later event	Duplicate events
False arrival claim	Geofence plus QR or beacon plus customer OTP	Audit review	Mismatched arrivals
Hub has no stock for a restock request	Hub marks it short; fills part, offers a substitute or points the rider to another hub	Kitchen production plan adjusted; an empty scooter leaves search	Short-fill rate
Many scooters reach reorder point together	Restocks are staggered; a cap limits scooters away per zone	Lowest-stock scooters go first; the rest wait	Scooters away per zone
Rider declines a suggested restock	The request expires and is drafted again at the next stock check	Ops sees the decline reason	Declines per shift
Duplicate or conflicting restock requests	Merged into one request per scooter	The hub sees one request	Merged requests
Hub is late past its queue slot	The slot is reassigned and the rider is told the new time	Use another hub, or wait	Hub wait time
Scooter stock hits zero during a busy spell	The scooter leaves search at once; open reservations are settled first	Send it to the hub; rematch nearby orders	Empty scooters in service
Wrong lot or temperature at hub handover	The rider's scan rejects the lot; the request stays open	Hub repacks; the lot is quarantined	Rejected handovers

Rules that hold in every scenario

Five rules decide what happens when the plan breaks.

1. **No promise without checks.** The 90 s offer appears only when stock, route, rider and gate checks pass; otherwise the app shows a longer ETA or says unavailable.
2. **Safety beats the clock.** A rider never speeds or skips a stop to save the timer, and is never penalised for a lawful, safe delay.
3. **One clock, server-owned.** Acceptance and arrival times are server-signed; no device can change them.
4. **Every miss has a remedy.** A missed or failed order triggers the stated refund, credit or replacement automatically.
5. **Repositioning is paid work.** Moving scooters to fix coverage counts as paid time, so riders have no reason to resist it.